Echoes Of The Forgotten Remastered

Alex Zamora Posadas
Xavier Guerrero
Minh Triet Le
Edwin Piedras
Beau Dougan

Software Design Specification

&

Project Delivery Report

Version: (1.1) **Date: (5/10/2025)**

Table of Contents

1.0 Introduction

When initially designing the game last semester, our goal was to create an older 8 Bit-style game with enhanced features to fully encapsulate everything a retro game wants. Using a simple *Canva* image as a starting point, we had broken down the general structure of the game into three major GUI components:

Mainbar: The key feature of the game. This will be responsible for displaying the story periodically as the player progresses through the story, showcasing the dialogue and combat system.

Optionbar: The user interaction. This showcases the player's dialogue options, allowing them to collect items, fight enemies, and even evade altercations entirely (if they're lucky).

Sidebar: The display of the user progression. This displays the players updated stats as they eventually gather items and immerse themselves in the combat.

After creating the story and the general format on the screen, we soon realized that if we really wanted this project to be a success, we were going to have to pull all the strings we could to make this game great. To do so, we looked at some solid reference material that would eventually become the cornerstone of the overall project. Inspired by well known games such as *Undertale* and *Final Fantasy*, we sought to utilize some of their core mechanics to add some much needed flair, adding that "wow" factor. First, we created a pixelated text class '8BitTestDisplay' that would read in from an alphabet list of two-dimensional arrays, displaying text in that iconic pixelated format seen in many classic RPGs to really bring home the classic feeling. Next, we revamped our story to have a less-generic tone as well as added some delay to the text to finalize what the 'Mainbar' would look like. Finally, we added a save/load feature that created an editable text file along with a read function that displayed the players old savepoint along with their updated stats. With the impending deadline, we had to scrap some of our niche features we had planned, focusing on the most important aspect: program functionality. Once everything was said and done, we begrudgingly wrapped up our work and presented our final product at the time.

To our surprise, we were given the chance to further improve our project this semester while also getting a fresh perspective from our newly established group of five. To do so, we had decided to implement some new software design patterns:

Singleton Pattern: Used for our newly added music feature. Allowed for only one instance of sound to be playing at all times.

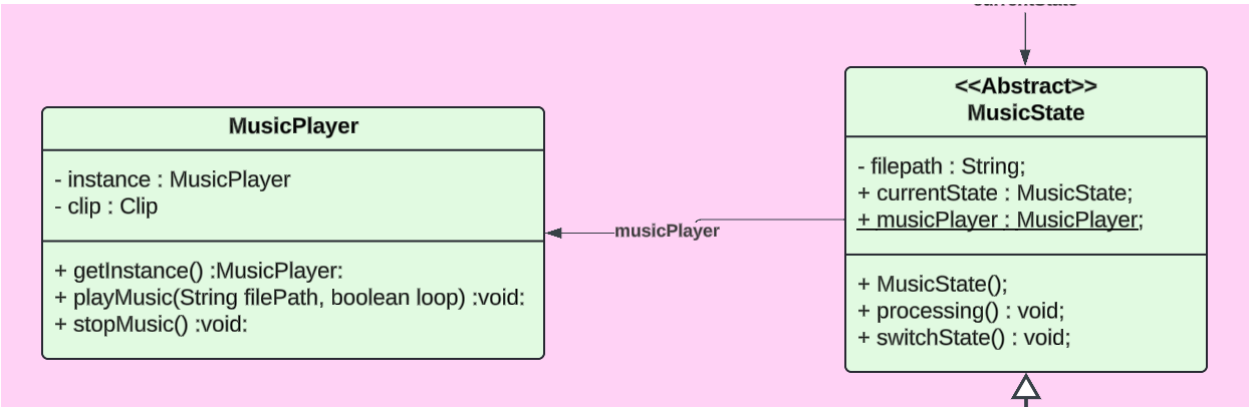
State Pattern: Separated internal game functionality into separate parts (combat state, music state, talking state, music state).

Snapshot Pattern: refactored our existing save/load state for better efficiency and clarity.

Decorator Pattern: Added enchantments to already existing objects dynamically for more item variety.

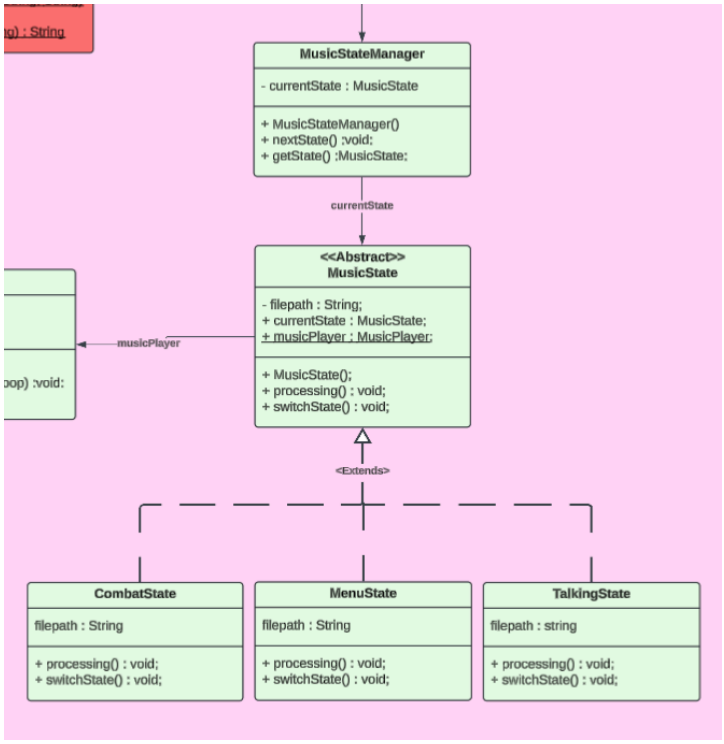
With our newly implemented system, we were able to add our niche features we wanted last semester such as having music running during game progression and conquered the text-wrapping visual bug that plagued us not so long ago. Overall, the new design patterns we learned this semester greatly influenced our project's success, resulting in proud smiles among the group.

2.0 System Design



Singleton:

We chose the singleton design pattern for the sound feature we added because we would never want to play two different songs at the same time. Using singleton ensures that there is only ever one audio player object at a time. This means instead of creating new audio objects every time it calls the get instance function which returns the existing one or creates one if there is no existing one. The MusicPlayer object which uses the singleton design pattern connects directly to the music state abstract class which controls which song is playing. This MusicPlayer object has 3 functions, one being the getInstance function which was explained above. Another function would be playMusic which takes in a string for the path to the music file and a boolean for whether or not the music should loop. For now all music states have a value of true for the loop parameter but the feature is there for if we change things in the future. The final function is the stopMusic function which stops the music and is not currently used but is implemented into the code for the future if we need to stop a song for a certain scene. This pattern is perfect for the audio we wished to add to the game because we will only use one MusicPlayer object at a time and if two were used at the same time it would detract from the user's experience greatly.



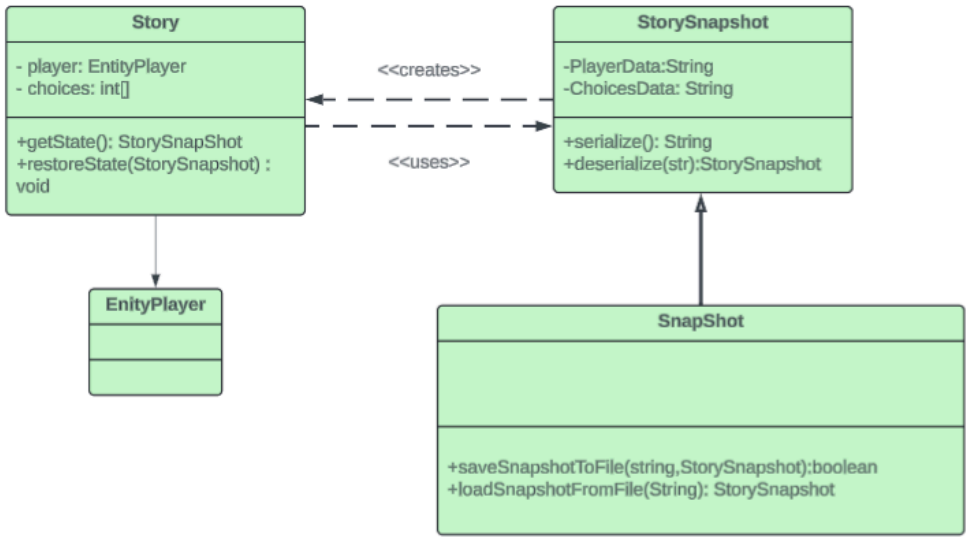
State:

We chose the state pattern because we wanted our music to change depending on what was happening in the game. The states are called through a MusicStateManager class which switches between music states. This manager class uses a MusicState object which is an abstract class for the three different game states. This object has three variables which are filepath for the song that matches the state. currentState which holds the current state of the game. And musicPlayer which is the music player object which is grabbed through getInstance() on the MusicPlayer class. It has a constructor and two void functions. One function called processing processes the current state and plays the correct song for what state the game is currently in. The other function is switch state which switches the state to the next in line. When switching to the next state in MusicStateManager it automatically calls processing since currently there should not be any downtime where music is not playing.

There are three different states used in the game which play three different songs. The menu state, combat state, and talking state are the three states we decided were most important to have different music. The menu state plays a song for the main menu. This song ends when the state is switched after the user enters their name and the game begins. When calling nextState when the state is currently for the menu the manager switches it to the talking state. The music then changes to the talking state which is when dialogue is printed onto the screen and the user is reading the text. When entering a battle nextState is called on the manager and when in the talking state this changes the current state to the combat state to play a more fast paced song to immerse the user in the battle. After the battle has ended the state is then switched back to the talking state through calling the next state function. The state function was used because in this game there are three very distinct periods of time where the current function of the game is relatively stable and these were the times where we wished to have

music fitting what was happening on the screen. Because these different states of the game transition between each other frequently we decided the state diagram was perfect for the purpose of playing fitting music that matched what was currently happening in the game.

Snapshot



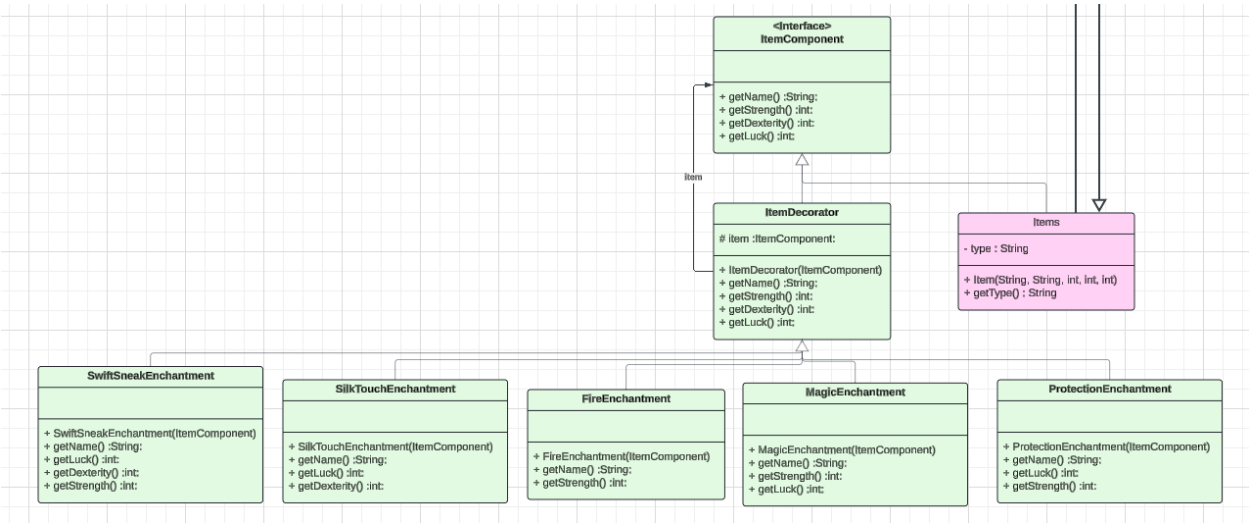
We chose the Snapshot (Memento) Pattern for our system because it directly addresses one of the most critical needs in a story-driven game: the ability to preserve and restore game state reliably and cleanly. In our design, the player's journey is heavily influenced by narrative choices, stat progression, and item acquisition — all of which must be remembered accurately to maintain narrative and gameplay coherence across play sessions.

The snapshot pattern was a natural fit because it allows us to encapsulate the full game state (such as player attributes, equipment, and story choices) into a single object (the StorySnapshot). This snapshot can then be saved and reloaded without tightly coupling storage logic to gameplay classes like EntityPlayer and Story. This decision helps us avoid scattered and brittle serialization code throughout the system, promoting better modularity and maintainability.

Additionally, we chose this pattern for its alignment with flexibility and future-proofing. As we expand the game with more chapters, story branches, and features (e.g., multiple save slots or undo mechanisms), the snapshot structure makes it easy to manage these extensions. The SnapShot class (formerly DAO) serves as a caretaker that interacts with persistent storage, keeping our business logic untouched.

In short, we adopted the snapshot pattern because it supports our design goals of clean separation of concerns, ease of data persistence, extensibility, and a better user experience — all of which are essential for a robust, choice-driven interactive narrative system.

Decorator



We chose the Decorator Pattern because we wanted more of a variety for our item objects. This was one of our niche features we wanted to add last semester but we ended up sticking with the standard item names we came up with on the fly to focus on functionality. Once we felt that the gameplay feature was fully developed, we used the decorator pattern to add more responsibility to item objects dynamically, while providing a fresh look to the somewhat plain item names we had previously, such as ‘iron axe’.

As the player progresses through the game, they will be prompted multiple times to get the chance to obtain items, each with a variety of different enchantments. As seen in the presentation video, the player will receive an ‘iron axe’ with a fire enchant added, which give the weapon an increased chance to inflict damage on enemies. The player will also get the chance to obtain other enchantments such as ‘swift sneak’ which increases the luck stat and ‘protection’ which increases the dexterity stat.

With these newly implemented enchantments, the decorator pattern adds that much needed layer of variety for game progression, creating a memorable experience for the player.

3.0 System Implementation

Snapshot

```
public class StorySnapshot {

    private final int[] choices;
    private final EntityPlayer player;

    public StorySnapshot(int[] choices, EntityPlayer player) {···

    public int[] getChoices() {···

    public EntityPlayer getPlayer() {···

    public String serialize() {···

    public static StorySnapshot deserialize(String data) {···
}
```

This class is the core of the Memento pattern. It encapsulates the complete state of the game needed to recreate a specific moment in gameplay.
It stores two primary pieces of information:

- An int[] choices array to capture the decisions the player has made through the story.
- An EntityPlayer player object representing the player's state at that point in the game.

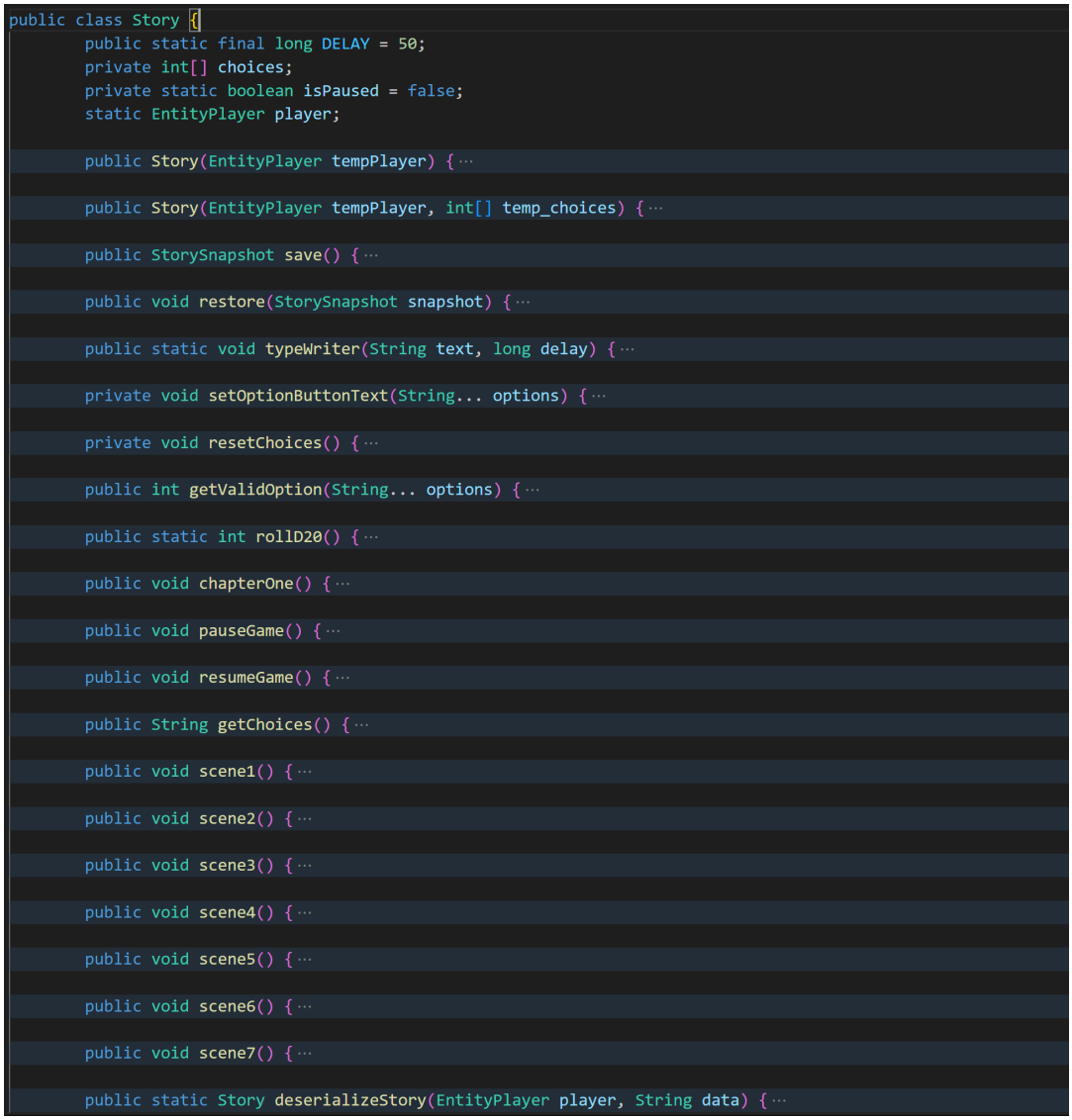
The class provides a constructor for creating snapshots, getChoices() and getPlayer() methods to retrieve state data, and serialize() and deserialize(String data) methods to convert the snapshot to and from a savable string format. These serialization methods are crucial for persistent storage.

```java
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

import model.StorySnapshot;

import java.io.FileWriter;

    public class SnapShot {

        public SnapShot() {…

        static public boolean checkExistingPlayerFile(String playerName) {…

        static public File CreatePlayerFile(String playerName) {…

        static public String ReadPlayerFile(String playerName) {…

        static public boolean writeToPlayerFile(String playerName, String text) {…

        static public String ReadItemFile(String ItemName) {…

        static public boolean saveSnapshotToFile(String playerName, StorySnapshot snapshot) {…

        static public StorySnapshot loadSnapshotFromFile(String playerName) {…

    }
```

This renamed DAO class now functions as the Caretaker in the Snapshot pattern. Its job is to manage saving and loading StorySnapshot instances without needing to understand the internal structure of the snapshot itself.
It contains methods such as:

- saveSnapshotToFile(String, StorySnapshot): Saves a serialized snapshot to a file.
- loadSnapshotFromFile(String): Reads from a file and reconstructs a StorySnapshot using deserialization.
- checkExistingPlayerFile(), ReadItemFile(), etc., are utility functions supporting the snapshot mechanism.

By decoupling snapshot creation/restoration from file I/O, the system remains modular and maintainable.



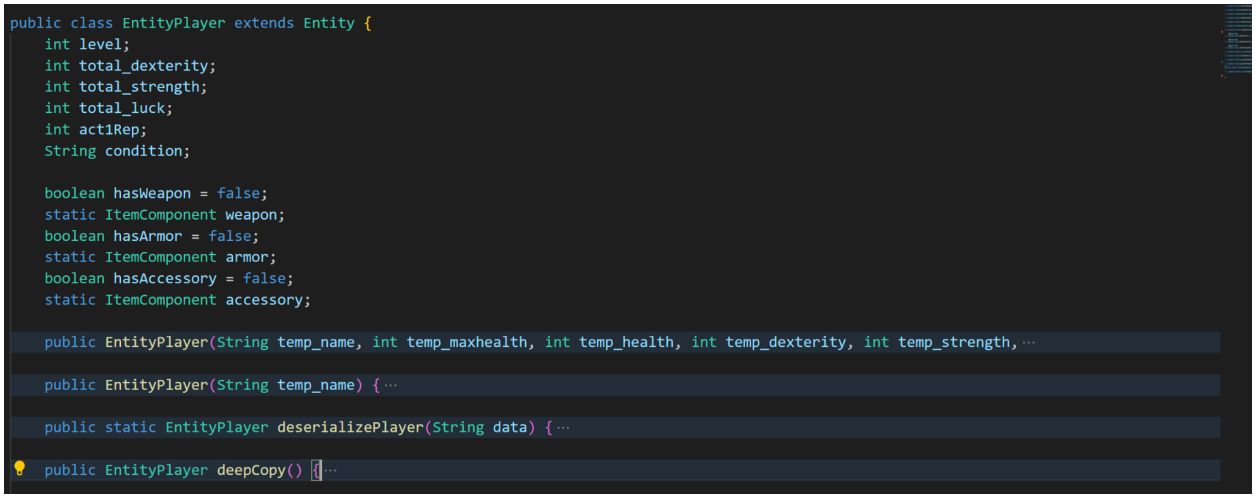
```
public class Story {
        public static final long DELAY = 50;
        private int[] choices;
        private static boolean isPaused = false;
        static EntityPlayer player;

        public Story(EntityPlayer tempPlayer) {···

        public Story(EntityPlayer tempPlayer, int[] temp_choices) {···

        public StorySnapshot save() {···

        public void restore(StorySnapshot snapshot) {···

        public static void typeWriter(String text, long delay) {···

        private void setOptionButtonText(String... options) {···

        private void resetChoices() {···

        public int getValidOption(String... options) {···

        public static int rollD20() {···

        public void chapterOne() {···

        public void pauseGame() {···

        public void resumeGame() {···

        public String getChoices() {···

        public void scene1() {···

        public void scene2() {···

        public void scene3() {···

        public void scene4() {···

        public void scene5() {···

        public void scene6() {···

        public void scene7() {···

        public static Story deserializeStory(EntityPlayer player, String data) {···
```

This class represents the Originator — the component whose state needs to be saved and restored. It encapsulates the game's narrative logic and player's progression through the choices[] array and the EntityPlayer.

Key methods:

- save(): Creates and returns a new StorySnapshot with the current player and choices[].
- restore(StorySnapshot snapshot): Reinstates the state of the story and player from a given snapshot.
- getChoices() and scene methods (e.g., chapterOne, scene1) represent the evolving narrative context that benefits from state capture.

With these methods, the Story class can delegate snapshot management to external classes (e.g., Game or SnapShot) without breaking encapsulation.



Although EntityPlayer is not part of the snapshot design pattern *per se*, it is the object whose state is included inside the StorySnapshot. For this reason, it must support:

- Deep copying (deepCopy()),
- Serialization (getPlayerData()),
- Deserialization (deserializePlayer(String)).

These methods allow the snapshot to reliably store and reconstruct the player's stats, equipment, and progression state — essential for a meaningful save system.

Decorator

```
313         }
314     */
315     public static ItemComponent parseItemComponent(String encoded) {
316         System.out.println("Item: " + encoded);
317         if (encoded.equals("null")) return null;
318 
319         String[] parts = encoded.split(":");
320         String baseName = parts[parts.length - 1];
321         ItemComponent item = getItem(baseName);
322 
323         for (int i = parts.length - 2; i >= 0; i--) {
324             switch (parts[i]) {
325                 case "FireEnchantment":
326                     item = new FireEnchantment(item);
327                     break;
328                 case "MagicEnchantment":
329                     item = new MagicEnchantment(item);
330                     break;
331                 case "ProtectionEnchantment":
332                     item = new ProtectionEnchantment(item);
333                     break;
334                 case "SilkTouchEnchantment":
335                     item = new SilkTouchEnchantment(item);
336                     break;
337                 case "SwiftSneakEnchantment":
338                     item = new SwiftSneakEnchantment(item);
339                     break;
340                 default:
341                     System.out.println("Unknown decorator: " + parts[i]);
342             }
343         }
344 
345         return item;
346     }
```

```
347 
348     public static ItemComponent applyEnchantments(ItemComponent base, String... enchantments) {
349         for (String ench : enchantments) {
350             switch (ench) {
351                 case "Fire":
352                     base = new FireEnchantment(base); break;
353                 case "Magic":
354                     base = new MagicEnchantment(base); break;
355                 case "Protection":
356                     base = new ProtectionEnchantment(base); break;
357                 case "SilkTouch":
358                     base = new SilkTouchEnchantment(base); break;
359                 case "SwiftSneak":
360                     base = new SwiftSneakEnchantment(base); break;
361             }
362         }
363         return base;
364     }
365 
366 
367 }
```

```java
package model;

public abstract class ItemDecorator implements ItemComponent {
    protected ItemComponent item;

    public ItemDecorator(ItemComponent item) {
        this.item = item;
    }

    public String getName() { return item.getName(); }
    public int getStrength() { return item.getStrength(); }
    public int getDexterity() { return item.getDexterity(); }
    public int getLuck() { return item.getLuck(); }
}
```

```java
package model;

public interface ItemComponent {
    String getName();
    int getStrength();
    int getDexterity();
    int getLuck();

}
```

```java
                        }
                }
                typeWriter("Rusty Sword acquired!+1 Attack", DELAY);
                player.setItem(EntityPlayer.getItem("rustySword"));
                typeWriter("Rusty Sword got Enchanted!!: / Fire Encahntment / + 2 Strength", DELAY);
                player.setItem(new FireEnchantment(player.getWeapon()));
                typeWriter("Rusty Sword got Enchanted AGAIN!!: / Magic Enchantment / +6 Strength / +4 Luck", DELAY);
                player.setItem(new MagicEnchantment(player.getWeapon()));
            }

```

This class is the core of the Decorator pattern.

The **Decorator Design Pattern** is used to dynamically add enchantments (like Fire, Magic, Protection) to items like (weapons, accessories, and armor) without modifying the base item classes. The “ItemComponent” interface defines the structure for all items, and “ItemDecorator” serves as a base wrapper that holds and delegates to another “ItemComponent”. Specific enchantments like “FireEnchantment” extend “ItemDecorator” to add extra behavior or stats.

This pattern allows flexible and modular composition of item features at runtime, avoiding the need for complex inheritance hierarchies with every possible combination.

Two key functions implement this:

- **parseItemComponent(String encoded)**: Reconstructs an enchanted item from a serialized string (e.g., "FireEnchantment:MagicEnchantment:Sword"), applying each decorator from innermost to outermost.

- **applyEnchantments(ItemComponent base, String... enchantments)**: Dynamically adds a sequence of enchantments to a base item by wrapping it in corresponding decorators.

Together, these functions demonstrate how the decorator pattern enhances flexibility and scalability in designing customizable, layered item behavior.

State

```java
package controller;

public class MusicStateManager {
    private static MusicState currentState;

    public MusicStateManager(){
        currentState = new MenuState();
        currentState.processing();
    }
    public static void nextState(){

        currentState = currentState.switchState();
        currentState.processing();
    }

    public static MusicState getState(){
        return currentState;
    }
}
```

```java
package controller;

public abstract class MusicState{
    private String filepath;
    public static MusicPlayer musicPlayer;

    public MusicState(){
        musicPlayer = MusicPlayer.getInstance();
    }

    public abstract void processing();

    public abstract MusicState switchState();
}
```

```java
package controller;

public class MenuState extends MusicState{

    String filepath = "music/MainMenuMusic.wav";
    @Override
    public void processing() {
        musicPlayer.playMusic(filepath, true);
    }


    @Override
    public MusicState switchState() {
        musicPlayer.stopMusic();
        return new TalkingState();

    }


}
```

```java
package controller;

public class TalkingState extends MusicState{
    String filepath = "music/Depression Shop.wav";
    @Override
    public void processing() {
        musicPlayer.playMusic(filepath, true);
    }

    @Override
    public MusicState switchState() {
        musicPlayer.stopMusic();
        return new CombatState();
    }

}
```

```java

package controller;
public class CombatState extends MusicState{

    String filepath = "music/SoulofCinder.wav";
    @Override
    public void processing() {

        musicPlayer.playMusic(filepath, true);
    }

    @Override
    public MusicState switchState() {
        musicPlayer.stopMusic();
        return new TalkingState();
    }


}
```

This class is the core of the State pattern, we modeled it as closely as we could from the slides that we went over in class . There isn't much meat and bones in this class, currently we just have 3 states; Menu, Talking, and Combat– this is why the states work with the musicPlayer class to play the appropriate music. This class works closely with the MusicPlayer class in order to actually add sound and music to the game.

Breakdown

- MusicManager works to keep track of the current state
- MusicState is our abstract class that all of our states implement
    - processing() - takes care of the actual music playing
    - switchState() - stops the music, and switches to the next state
- CombatState takes care of the combat music
- MenuState takes care of the menu music
- TalkingState takes care of the dialogue music

Singleton

```java
package controller;

import javax.sound.sampled.*;
import java.io.File;
import java.io.IOException;


public class MusicPlayer{
    private static MusicPlayer instance;
    private Clip clip;

    private MusicPlayer(){}

    public static MusicPlayer getInstance(){
        if(instance == null){
            instance = new MusicPlayer();
        }
        return instance;
    }

    public void playMusic(String filePath, boolean loop){
        try{
            if(clip != null && clip.isRunning()){
                clip.stop();
                clip.close();
            }
            AudioInputStream audioIn = AudioSystem.getAudioInputStream(new File(filePath));
            clip = AudioSystem.getClip();
            clip.open(audioIn);

            FloatControl gainControl = (FloatControl) clip.getControl(FloatControl.Type.MASTER_GAIN);
            if (gainControl != null) {
                // Decrease volume by 20 decibels
                gainControl.setValue(-20.0f);
            } else {
```

```
            } else {
                System.out.println("Master gain control not found");
                return;
            }

            if(loop){
                clip.loop(Clip.LOOP_CONTINUOUSLY);
            }
            else{
                clip.start();
            }
        } catch(UnsupportedAudioFileException | IOException | LineUnavailableException e){
            e.printStackTrace();
        }
    }

    public void stopMusic(){
        if(clip != null && clip.isRunning()){
            clip.stop();
        }
    }

}
```

The Singleton Design Pattern is used to ensure that only one instance of the “MusicPlayer” class exists throughout the application. This pattern is useful when a single, shared resource is needed globally — in this case, a centralized controller for background music playback.

- The class has a private static instance variable (instance) and a private constructor, preventing direct instantiation from outside.

- The “getInstance()” method lazily creates the “MusicPlayer” object the first time it's needed and returns the same instance every time after.

This guarantees that all parts of the program interact with the same music player, avoiding issues like overlapping audio clips or conflicting controls.

4.0 External Learning Resources

RyiSnow. *How to Make a 2D Game in Java #1 - The Mechanism of 2D Games*. YouTube, 3 Oct. 2021, https://www.youtube.com/watch?v=om59cwR7psI&list=PL_QPQmz5C6WUF-pOQDsbsKbaBZqXj4qSq.

"How to Play an Audio File Using Java." GeeksforGeeks, 5 June 2017, www.geeksforgeeks.org/play-audio-file-using-java/

5.0 Lessons Learned

Minh Triet Le

Throughout this project, I learned the importance of first identifying the core functionalities we want in our software. By clearly defining these early on, we can more effectively determine which design patterns best support each feature, leading to cleaner, more organized, and maintainable code.

Beau Dougan

In this project I learned important lessons on building onto existing codebases I had not previously worked on. This will probably prove very useful in my future career because most places I will work at will already have existing codebases that I will need to learn quickly to add things to it. I also learned how to add sound to java applications in addition to learning about what audio formats work with AudioInputStream and how to format a song from mp3 to wav to work with java.

Xavier Guerrero

I am really excited that we got the chance to go back to this project. Even a semester later, I still feel that this project concept was the most interesting compared to the others. I personally learned the importance of the 'O' principle for maintaining a codebase. To explain, we discovered that the class '8BitTestDisplay', the class that was responsible for displaying the pixelated text, was causing the majority of bugs we encountered. I should have ensured that there was less dependency with this class, allowing for extension but not modification.

Alex Zamora Posadas

This project really opened my eyes to making sure you start with a good design for your code so that you can make it easier to expand on in the future. Making code that just works is very different from making code that works and is good, this project really highlighted that fact. It also allowed us to apply coding conventions such as the SOLID principles, and gave us practice with the new patterns that we learned throughout the whole semester. Working with our already existing code base was a lot more fun too, since we didn't really have to worry about making something completely from scratch. Overall, I learned a lot through this project and had a fun time working on it again.

Edwin Piedras

Throughout this semester I wasn't sure what to expect from the final project because I didn't know if I was still going to have to work on the same project from SE 370. When we formed the teams for the project I was glad my team from last semester was still willing to continue working on our 2d based video game. So it became more interesting and exciting to

visualize what else we can do to improve this project. Definitely the idea of having to implement four new design patterns from this current semester was a great way to apply our knowledge into the project. It gave us a sense of understanding on how these architectural patterns can actually be implemented into our code and implemented in various ways to our project. Overall a lesson learned from this whole project is the idea of teamwork and the consistency of gathering up online or in person and coming up with ideas to approach this project was the greatest lesson I learned. How important it is to work as a team and collaborate about any minor details to the project.